

Patchscopes: A Unifying Framework for Inspecting Hidden Representations of Language Models

Asma Ghandeharioun^{*1} Avi Caciularu^{*1} Adam Pearce¹ Lucas Dixon¹ Mor Geva^{1,2}

Abstract

Understanding the internal representations of large language models (LLMs) can help explain models’ behavior and verify their alignment with human values. Given the capabilities of LLMs in generating human-understandable text, we propose leveraging the model itself to explain its internal representations in natural language. We introduce a framework called *Patchscopes* and show how it can be used to answer a wide range of questions about an LLM’s computation. We show that many prior interpretability methods based on projecting representations into the vocabulary space and intervening on the LLM computation can be viewed as instances of this framework. Moreover, several of their shortcomings such as failure in inspecting early layers or lack of expressivity can be mitigated by *Patchscopes*. Beyond unifying prior inspection techniques, *Patchscopes* also opens up *new* possibilities such as using a more capable model to explain the representations of a smaller model, and multihop reasoning error correction¹.

1. Introduction

The question of what information is captured in the hidden representations of large language models (LLMs) is of key importance in control and understanding of modern generative AI, and has drawn substantial attention recently (Casper et al., 2022; Madsen et al., 2022; Patel & Pavlick, 2021; Nanda et al., 2023). To tackle this question, prior work has introduced a diverse array of interpretability methods, which largely rely on three prominent approaches: training linear

^{*}Equal contribution ¹Google Research ²Tel Aviv University. Correspondence to: Asma Ghandeharioun <aghandeharioun@google.com>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024. Copyright 2024 by the author(s).

¹Code is publicly available at <https://pair-code.github.io/interpretability/patchscopes>.

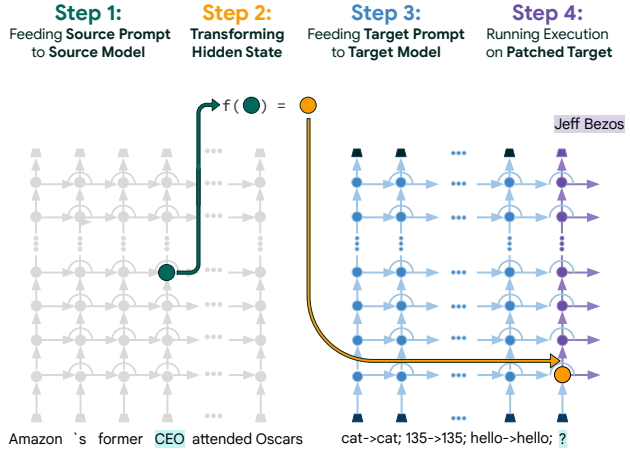


Figure 1. Illustration of our framework, showing a *Patchscope* for decoding the hidden representation of “CEO” in the source prompt (left). We use a target prompt (right) comprised of few-shot demonstrations of string repetitions to encourage the LLM to explain its internal representation. **Step 1:** Run the forward computation on the source prompt in the source model. **Step 2:** Optionally transform the hidden state at the source layer. **Step 3:** Run the forward computation on the target prompt up to the target layer in the target model. **Step 4:** Patch the target representation of the token “?” at the target layer with the transformed representation (from step 2), then continue the forward computation from that layer onward. The modularity of *Patchscopes* allows designing a variety of methods by configuring the target prompt, model and transformation.

classifiers, called probes, on top of hidden representations (Belinkov & Glass, 2019; Belinkov, 2022; Alain & Bengio, 2017), projecting representations to the model’s vocabulary space (nostalgebraist, 2020; Din et al., 2023; Belrose et al., 2023), and intervening on the computation to identify if a representation is critical for certain predictions (Meng et al., 2022a; Wallat et al., 2020; Wang et al., 2022; Conmy et al., 2023; Geva et al., 2023).

Despite the wide success of these methods, they each exhibit practical shortcomings. First, probing relies on supervised training for pre-defined classes, which is hard to scale when there is a large number of classes or when all the categories are not known a priori. Second, the accuracy of vocabulary

Table 1. Many prior inspection methods with various objectives can be viewed as Patchscopes. The rows highlighted in green show Patchscope configurations that overcome several limitations of prior methods through more expressive inspection that is training-data free and is more robust across layers.

Inspection Objective		Expressive	Training Data Free	Robust Across Layers
Inspecting Output Distribution	Few-Shot Token Identity Patchscope (§4.1)	✓✓	✓	✓✓
	Logit Lens (nostalgebraist, 2020), Embedding Space Analysis (Dar et al., 2023)	✓	✓	✗
	Tuned Lens (Belrose et al., 2023)	✓	For learning mappings	✓
	Future Lens (Pal et al., 2023)	✓	For learning mappings	✓✓
Feature Extraction	Zero-Shot Feat. Ext. Patchscope (§4.2)	✓✓	✓	✓✓
	LRE Attribute Lens (Hernandez et al., 2023b)	✓	For linear relation approx.	✓✓
	Probing (e.g., Belinkov & Glass, 2019; Belinkov, 2022; Alain & Bengio, 2017; Wang et al., 2023)	✗	For training probe	✓
Entity Resolution	Entity Description Patchscope (§4.3)	✓✓	✓	✓✓
	X-Model Entity Desc. Patchscope (§4.4)	✓✓✓	For learning mappings	✓✓
	Causal Tracing (Meng et al., 2022a)	✗	✓	✓✓
	Attention Knockout (Wang et al., 2022; Conmy et al., 2023; Geva et al., 2023)	✗	✓	✓✓
Inspection Application	Early Exiting, e.g., Linear Shortcuts (Din et al., 2023)	✓	For learning mappings	✓
	Caption Generation, e.g., Linear Mapping (Merullo et al., 2022)	✓	For learning mappings	✓

projections substantially decreases in early layers and the outputs are often hard to interpret. Last, all the above methods are not as expressive as one might like: they provide class probabilities or most likely tokens, as opposed to a high-quality explanation in natural language.

In this work, we argue that the advanced capabilities of LLMs in generating human-like text can be leveraged for “translating” the information in their representations for humans. We introduce a modular framework, called Patchscopes (§3), that can be configured to query various kinds of information from LLM representations. Patchscopes decode specific information from a representation within an LLM by “patching” it into the inference pass on a different prompt that has been designed to encourage the extraction of that information.² Such configuration (a Patchscope) can be viewed as an inspection tool geared towards a particular objective, as illustrated in Fig. 1.

We show that many existing methods, including those that rely on vocabulary projections and computation interventions, can be cast as Patchscopes. Moreover, new configurations of our framework introduce more effective tools in addressing the same questions, while mitigating several limitations of prior approaches. Also, Patchscopes enables addressing underexplored questions, such as fine-grained analysis of the input contextualization process and the extent to which a more expressive model can be used to inspect hidden representations of a smaller model.

²While patching (or “activation patching”) by itself is not a new technique, to the best of our knowledge, we are the first to propose using it for decoding information from hidden representations in a configurable and expressive manner. See more details in §3.2.

We conduct a series of experiments to evaluate the benefits and opportunities introduced by Patchscopes, focusing on auto-regressive LLMs. First, we consider the problem of estimating the model’s next-token prediction from its intermediate representations (see §4.1). Across multiple LLMs, we show that using a few-shot token identity prompt, a prompt in the form of “tok₁ → tok₁; tok₂ → tok₂; ...; tok_k” where tok_i refers to a random token, leads to substantial gains over vocabulary projection methods. Next, we evaluate how well Patchscopes can decode specific attributes of an entity from its LLM representations, when these are detached from the original context (see §4.2). We observe that, despite using no training data, Patchscopes significantly outperforms probing in six out of twelve commonsense and factual reasoning tasks, and works comparably well in all but one of the remaining six.

Beyond output estimation and attribute decoding, Patchscopes can address questions that are hard to answer with existing methods. In §4.3, we apply Patchscopes to study how LLMs contextualize input entity names in early layers, where vocabulary projections mostly fail and other methods, at best, provide only a binary signal of whether the entity has been resolved (Youssef et al., 2023; Tenney et al., 2019). With a new Patchscope, we are able to *verbalize* the gradual entity resolution process. For example, we show that, as the model processes the final token of “Alexander the Great” throughout the layers, it reflects different entities starting from “Great Britain”, to “the Great Depression”, to finally resolving “Alexander the Great”. Then, in §4.4 we show how one can further improve Patchscope expressivity by using a stronger target model, e.g., Vicuna 13B instead of Vicuna 7B.

Lastly, we showcase the utility of Patchscopes for fixing latent multi-hop reasoning errors, particularly when the model is capable of conducting each reasoning step correctly, but fails when they need to be composed in-context (§5). Building on top of the data provided by Hernandez et al. (2023b), we introduce a more complex task that requires two steps of factual reasoning. Patchscope achieves 50% accuracy on this task, outperforming chain-of-thought (Wei et al., 2022) (35.71%) and vanilla generations (19.57%).

To conclude, our work makes the following contributions: We propose Patchscopes, a general modular framework for decoding information from the hidden representations in LLMs. We show that prominent interpretability methods can be viewed as instances of Patchscopes, and new configurations result in more expressive, robust across layers, and training-data free alternatives that mitigate their shortcomings. In addition, novel configurations introduce unexplored possibilities of stronger inspection techniques, as well as practical benefits, such as correcting multi-hop

reasoning errors.

2. Related Work

Activation patching is a causal intervention, commonly used as a tool for studying if certain activations play a key role in a model’s computation (Geiger et al., 2021; Vig et al., 2020). Patching has been used largely for localizing specific information to specific layers and token positions (Goldowsky-Dill et al., 2023; Meng et al., 2022a;b; Stolfo et al., 2023; Merullo et al., 2023), and for finding information propagation paths in the computation (Wang et al., 2022; Geva et al., 2023; Hendel et al., 2023; Hanna et al., 2023; Lieberum et al., 2023). Prior works have also used specific forms of cross-model patching called stitching, in non-transformer architectures, mostly to analyze representational similarity (e.g., Bansal et al., 2021; Csiszárík et al., 2021; Lenc & Vedaldi, 2015). Despite certain limitations (Hase et al., 2023; Zhang & Nanda, 2023), patching remains a principal tool for mechanistic interpretability (Conmy et al., 2023).

Given promising results from emerging interpretability efforts that employ LLMs to generate human-like text for inspection (e.g., Mousi et al., 2023; Slobodkin et al., 2023; Bills et al., 2023), we argue that using patching only for localization purposes is myopic, and propose to use it for “translating” LLM representations into natural language. Very recently, patching has been used to study new problem setups (e.g., Pal et al., 2023; Hernandez et al., 2023b), all of which can be seen as different configurations of our proposed framework (see §3.2).

Among the growing research efforts in inspecting hidden representations of neural networks, probing classifiers are perhaps the most common (e.g., Alain & Bengio, 2017; Belinkov & Glass, 2019; Belinkov, 2022; Wang et al., 2023), and methods using projections into the vocabulary space or their extensions to other domains are another key category (e.g., Merullo et al., 2023; Geva et al., 2022b; nostalgebraist, 2020; Belrose et al., 2023; Dar et al., 2023; Din et al., 2023; Langedijk et al., 2023; Vilas et al., 2023). While various other latent inspection methods exist (e.g., Zhou et al., 2018; Strobelt et al., 2017; Ghandeharioun et al., 2021; Kim et al., 2018), the above are the most relevant to this work.

3. Patchscopes

In this section, we introduce Patchscopes and show how it extends prior interpretability methods with new capabilities. While not limited to particular LLM architectures, this work focuses on auto-regressive transformer-based LLMs.

3.1. Framework Description

The key idea in Patchscopes is to leverage the advanced capabilities of LLMs to generate human-like text for “translating” the information encoded in their own hidden representations. Concretely, given a hidden representation obtained from an LLM inference pass, we propose to decode specific information from it by *patching* it into a different inference pass (of the same or a different LLM) that encourages the translation of that specific information.

Notably, the rest of the forward computation after patching can augment the representation with additional information, hence, this approach does not guarantee that the patched representation itself stores *all* that information. However, dispatching the representation from its original context (the source prompt) stops contextualization and guarantees that no further information from the source prompt is incorporated in the post-patching computation. Thus, our framework reveals if specific information *can be decoded from the patched representation via the post-patching computation*.

Given an input sequence of n tokens $S = \langle s_1, \dots, s_n \rangle$ and a model $\mathcal{M}$ with L layers, $\mathbf{h}_i^\ell$ denotes the hidden representation obtained at layer $\ell \in [1, \dots, L]$ and position $i \in [1, \dots, n]$, when running $\mathcal{M}$ on S . To inspect $\mathbf{h}_i^\ell$, we consider a separate inference pass of a model $\mathcal{M}^*$ with L^* layers on a target sequence $T = \langle t_1, \dots, t_m \rangle$ of m tokens. Specifically, we choose a hidden representation $\bar{\mathbf{h}}_{i^*}^{\ell^*}$ at layer $\ell^* \in [1, \dots, L^*]$ and position $i^* \in [1, \dots, m]$ in the execution of $\mathcal{M}^*$ on T . Moreover, we define a mapping function $f(\mathbf{h}; \theta) : \mathbb{R}^d \mapsto \mathbb{R}^{d^*}$ parameterized by θ that operates on hidden representations of $\mathcal{M}$, where d and d^* denote the hidden dimension of representations in $\mathcal{M}$ and $\mathcal{M}^*$, respectively. This function can be the identity function, a linear or affine function learned on task-specific pairs of representations, or even more complex functions that incorporate other sources of data. The *patching* operation refers to dynamically replacing the representation $\bar{\mathbf{h}}_{i^*}^{\ell^*}$ during the inference of $\mathcal{M}^*$ on T with $f(\mathbf{h}_i^\ell)$. Namely, by applying $\bar{\mathbf{h}}_{i^*}^{\ell^*} \leftarrow f(\mathbf{h}_i^\ell)$, we intervene on the generation process and modify the computation after layer ℓ^* .

Overall, a Patchscope intervention applied to a representation determined by $(S, i, \mathcal{M}, \ell)$, is defined by a quintuplet $(T, i^*, f, \mathcal{M}^*, \ell^*)$ of a target prompt T , a target position i^* in this prompt, a mapping function f , a target model $\mathcal{M}^*$, and a target layer ℓ^* of this model. It is possible that $\mathcal{M}$ and $\mathcal{M}^*$ are the same model, S and T are the same prompt, and f is the identity function $\mathbb{I}$ (i.e., $\mathbb{I}(\mathbf{h}) = \mathbf{h}$). Next, we show how this formulation covers prior interpretability methods and further extends them with new capabilities.